

INVENTORYAI

Never run out, never overstock: a plain-English guide to what this product does and why

Document	InventoryAI, Product Requirements
Product Version	v1.0.0
Document Version	1.0
Status	Draft
Date	July 25, 2026
Author	Satyam Pandey
Repository	github.com/SatyamPandey07/inventory-forecasting-optimization
Live Demo	fascinating-narwhal-5f1ac9.netlify.app
License	MIT

Start Here, No Jargon

Every store or warehouse has the same two-sided headache. Order too little of something, and you run out and lose the sale. Order too much, and the extra stock sits on a shelf tying up cash, taking up space, and sometimes going stale or out of season before it sells. Get this balance right for one product and it's easy. Get it right for thousands of products at once, each with its own sales pattern, is a full-time job for a whole team.

InventoryAI is software that does that job automatically. It watches how a product actually sells, predicts what's coming over the next three months, works out the smartest amount to order and when, tests that decision against bad luck scenarios like a late shipment, and then explains the whole thing in a sentence a busy manager can read in five seconds. It's built as a real, working product, not a prototype: five separate services running together, a live demo anyone can click into, and a full monitoring setup watching its own health.

The Two-Sided Problem, in Plain Words

The problem	What it actually costs
Running out of stock	A customer wants to buy something, it isn't there, and the sale is simply gone. Multiply that by every product, every week, and it adds up fast.
Sitting on too much stock	Money that's tied up sitting in a warehouse can't be used anywhere else. On top of that, storage space, insurance, and handling all cost real money, and some stock ages out or goes out of season before it ever sells.

InventoryAI's whole job is finding the number in between these two costly mistakes, for every product, automatically, every single week.

Who This Is Actually For

- **Supply chain directors.** The people accountable for the big picture: total inventory cost, service levels, and whether the business is quietly losing money to stockouts or overstock.
- **Operations managers.** The people who act on the day-to-day recommendations, deciding what actually gets ordered and when.
- **Anyone managing supplier relationships.** Since the product also tracks how reliable each supplier has been, which feeds back into how much safety stock gets held for their products.

A Day in the Life

The product is organized as four screens, and walking through them in order is the easiest way to understand what it actually does.

First step: the Executive Control Tower

This is the homepage. It's a live view of everything that matters right now: how many products are being tracked, what's sold in the last 30 days, how much revenue that represents, and a list of anything at risk of running out soon. It refreshes itself every five minutes, and the whole thing can be exported to a spreadsheet with one click.

Second step: AI Reorders and Recommendations

This is where the product actually tells someone what to do. Each card covers one product and says, in plain language, how much to order, why, and what it's worth. The README's own example: a recommendation that says it would save \$1,850 by avoiding a stockout. Every card has an Accept or Reject button, so a person always makes the final call, the system never orders anything on its own.

Third step: the What-If Simulator

This is the playground for nervous what-ifs. Drag a slider to say a supplier might be nine days late, or that demand might suddenly spike by 37 percent, and the system runs that scenario one thousand times over to show the realistic range of what could happen, not just a single guess.

Fourth step: Forecast Analytics

This is the report card. It shows how accurate the predictions have actually been over time, using a plain error percentage (the README's example shows a 4.5 percent error rate), and it tracks the running total of money saved by following the system's advice, an example figure of \$5,450 saved shows up in the README as a sample.

The Five Things It Actually Does

Underneath those four screens, there are five distinct engines doing the real work. Each one solves a specific, named problem, explained here without the statistics jargon.

1. Guessing what happens next

This is demand forecasting. The system looks at a product's sales history along with outside signals such as the season, upcoming holidays, and even the weather, and predicts demand for the next 90 days. It doesn't give one single number either, it gives a likely range, so a manager knows not just the best guess but how much that guess could be wrong by. Under the hood this runs on Facebook's open-source Prophet forecasting library.

2. Working out how much to order, and when

Once you know roughly what's coming, the next question is a math problem: what's the cheapest way to always have enough stock without holding too much? The system calculates three numbers for each product: how much to order at once, how much extra buffer stock to keep just in case, and the exact stock level that should trigger a new order. It also respects real supplier constraints, like a minimum order size or how long shipping actually takes. This calculation is what's formally called Economic Order Quantity, or EOQ, and it's solved using SciPy's numerical optimization tools.

3. Explaining all of that like a person would

Numbers and percentages don't sell a decision to a busy manager, a clear sentence does. This is where Anthropic's Claude AI comes in: it takes everything the system knows (the forecast, the weather outlook, upcoming holidays, and even what competitors are doing) and turns it into a short, plain-English recommendation someone can act on in seconds without needing to understand any of the statistics behind it.

4. Stress-testing the plan against bad luck

Forecasts are never perfect, and shipments don't always arrive on time. So the system runs a kind of rehearsal: it simulates the next 90 days one thousand separate times, each time with slightly different random luck (a late shipment here, a demand spike there), and looks at the spread of outcomes. This is called a Monte Carlo simulation, and it's the same basic idea as a weather forecast telling you there's a 30 percent chance of rain, except here it's telling you the range of costs a decision might realistically lead to.

5. Watching its own health

Software that makes financial recommendations needs to be trustworthy, so the system constantly checks itself. It tracks its own forecasting accuracy over time, logs whether people actually accept or reject its recommendations, and exposes all of this through a real monitoring setup (five dashboards built in Grafana) so a technical team can see at a glance if something is drifting or broken.

How the Pieces Talk to Each Other

None of this lives in one single program. It's split into five cooperating pieces, each doing one job well, which is a common way to build software that needs to stay reliable as it grows.

The five pieces, and how they talk to each other

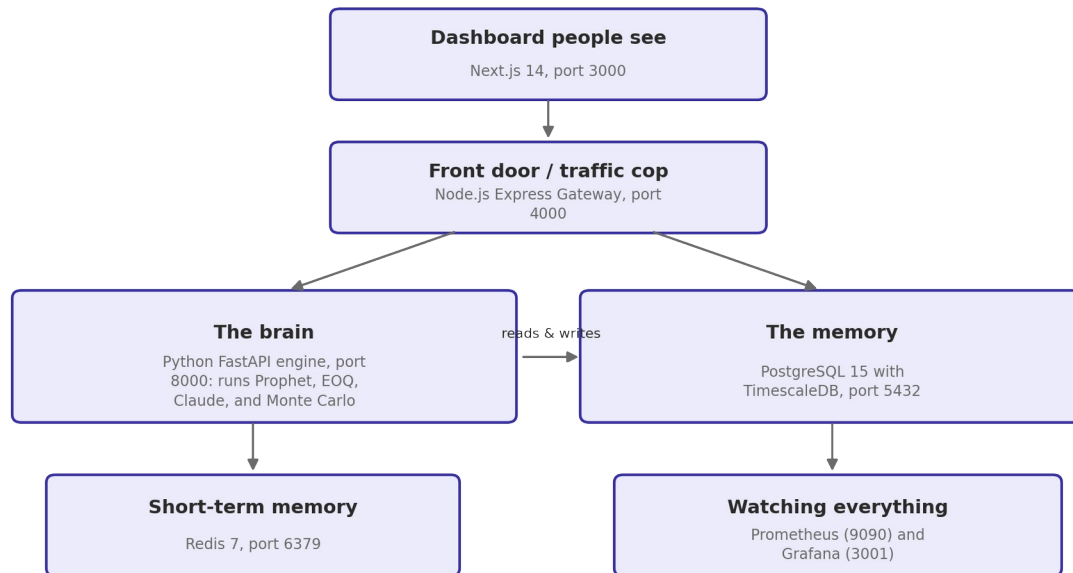


Figure 1. The five services that make up the system.

Piece	Job	Built with
Dashboard	What a person actually looks at and clicks	Next.js 14, React, TypeScript
Gateway	Checks who you are and routes your request to the right place	Node.js, Express
The brain	Runs the forecasting, math, AI reasoning, and simulations	Python, FastAPI
The memory	Stores sales history, forecasts, and recommendations permanently	PostgreSQL 15 with TimescaleDB
Short-term memory	Remembers recent calculations so it doesn't redo expensive work every time	Redis 7

TimescaleDB is a version of the regular PostgreSQL database built specifically to store huge amounts of time-stamped data, exactly what years of daily sales history looks like. Redis, the short-term memory, keeps a saved copy of each product's forecasting model and caches the AI's written explanations for an hour, so asking the same question twice in a row doesn't mean paying for and waiting on the same expensive work twice.

How One Recommendation Actually Gets Made

It helps to see the five engines above as one continuous assembly line rather than five separate features. Here's that same pipeline, drawn out as a single flow.

How one recommendation gets made

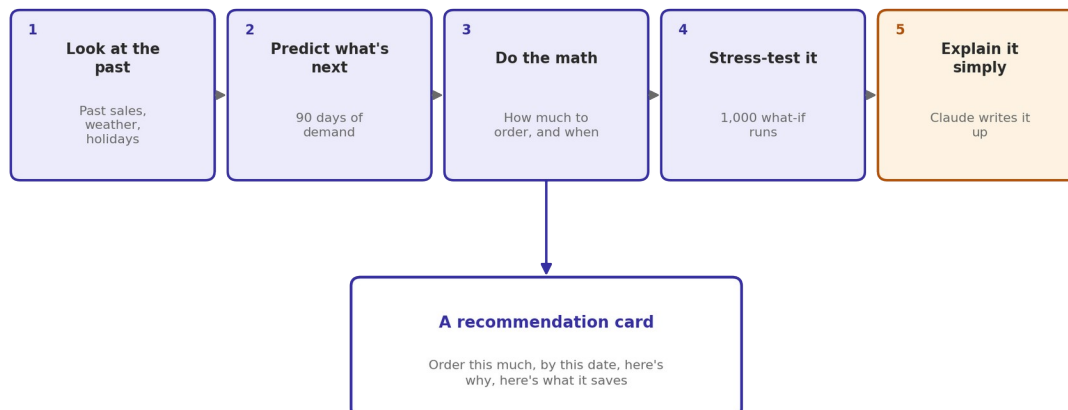


Figure 2. From raw sales history to a recommendation someone can act on.

Nothing in this pipeline skips a step. The AI explanation step, in particular, always runs after the math, not instead of it, so the plain-English sentence a manager reads is always backed by an actual calculation, not just a guess dressed up in nice language.

Logging In, and Staying in Your Own Lane

This is built to be used by more than one company at once, which brings up an important, unglamorous requirement: company A should never, under any circumstance, see company B's sales data, forecasts, or recommendations. In the industry this is called multi-tenancy, and it's enforced in a very literal way here: every single table in the database carries an organization ID, and every request checks that ID before returning anything.

- Sign up and sign in with email and password, with sessions kept secure and encrypted.
- Single sign-on support for Google, GitHub, and enterprise SAML 2.0, for companies that already manage logins centrally.
- A one-click Demo Admin login, so anyone evaluating the product can look around immediately without needing real credentials or a live database.
- Every API call and every database query is scoped to the logged-in user's own organization. There's no path, accidental or otherwise, for cross-company data to leak.

Keeping the Lights On

A forecasting system is only useful if it stays accurate and stays running, so there's a real operational routine behind it rather than a one-time setup.

What gets watched

Alert	What it means
Forecast accuracy low	Fires automatically if the prediction error rate creeps above 30 percent (in other words, accuracy drops below 70 percent), a signal the model needs attention.

Alert	What it means
Stockout detected	Fires the moment an actual stockout happens, not just a predicted one.
High API latency	Fires if typical response times start creeping past one second, an early warning that something is slowing down before people start to notice.

What happens automatically, on a schedule

- Every Monday at midnight UTC, the forecasting models retrain themselves on the latest data, rather than slowly going stale.
- A new model only replaces the old one if it's actually more accurate. This is often called a champion and challenger setup, and it means accuracy can only improve, never quietly get worse from an automatic update.
- Every Sunday at 2 AM UTC, forecast records older than a year are cleaned out automatically, so the database doesn't grow forever with data nobody needs anymore.

The One Feature That's Deliberately Not Fully Built Yet

One capability is worth calling out specifically, because the project's own documentation is unusually candid about it: automatically tracking competitor prices. Today, that data can only come in through manual entry or a webhook that a custom script could push data into. Fully automating it, meaning actually scraping prices off sites like Amazon, Walmart, and Target, runs into real obstacles.

- Those sites actively block automated crawlers using anti-bot systems like Cloudflare and DataDome.
- Modern retail sites render their prices using JavaScript after the page loads, so a simple crawler that just reads raw HTML won't even see the price.
- Prices often vary by region, so the same product page can quietly show a different number depending on where the request appears to come from.

The project's own documentation recommends solving this with a service called Bright Data, which handles the proxy and anti-bot problem so the product doesn't have to build that infrastructure itself. It comes with a real, known cost, which is worth planning around honestly rather than treating as free.

Scale	Roughly how much scraping	Estimated monthly cost
Starter	10,000 requests	About \$15 a month
Growing SaaS	100,000 requests	About \$120 a month
Enterprise	1,000,000 requests	Roughly \$500 to \$850 a month

What's Deliberately Left Out, For Now

- No automatic competitor price scraping yet, as covered above. Only manual entry and webhook ingestion exist today.
- No support for multiple warehouses coordinating with each other. Each product's numbers are calculated on their own, not as part of a shared network.

- No neural network based forecasting yet. Today's predictions come from Prophet, a solid statistical approach, rather than deep learning models.
- No mobile app. It's a responsive web dashboard, meant to work on a phone browser but not built as a native app.

What Needs to Be True for This to Work Well

- The forecasting engine needs enough sales history per product to find a real pattern. A brand new product with no history is a harder case than one with a year of data behind it.
- The gateway and the brain need to keep talking to each other quickly. If that connection slows down, the high-latency alert should catch it before customers do.
- Redis needs to stay up and healthy, since it's quietly doing a lot of the work of avoiding repeated, expensive recalculations.
- Everything assumes a company is comfortable connecting real sales data to a third-party AI service (Claude) to generate the plain-English explanations.

How We'd Know It's Actually Working

What we're watching	What good looks like	How we'd check
Forecast accuracy	Error rate stays comfortably under the 30 percent alert threshold for most products	The MAPE and MAE trends already tracked on the Forecast Analytics screen
Recommendations get used	A healthy share of recommendation cards get Accepted rather than ignored or rejected	Logged accept and reject decisions, already part of the audit trail
Real savings show up	The tracked savings total keeps climbing over time, not just in the first week	The aggregate savings figure shown on Forecast Analytics
The system stays healthy	Alerts fire rarely, and when they do, they get resolved quickly	Grafana dashboards and Prometheus alert history

Where This Could Go Wrong

Risk	Why it matters	What helps
A forecast quietly becomes inaccurate	Bad predictions lead directly to bad ordering decisions and real financial cost	The weekly retrain schedule, the champion versus challenger check, and the forecast accuracy alert all catch this before it goes unnoticed for long
Cross-company data isolation fails	Since this serves multiple companies at once, any leak between organizations is a serious trust and legal problem	Every table and every query is scoped to an organization ID as a strict, non-negotiable rule
People stop trusting the AI's	If the plain-English explanations	Recommendations are always

Risk	Why it matters	What helps
recommendations	feel wrong or generic even once, people quietly go back to guessing on their own	grounded in the actual math (forecast plus EOQ plus simulation), and every decision is logged for later review
Competitor price data stays manual longer than expected	Without automated scraping, that signal depends on someone remembering to enter it	The manual entry and webhook path keeps it usable today, with Bright Data integration documented as the clear next step

Where This Is Headed

The project's own roadmap lays out four clear expansion areas, each with a rough timeframe attached.

When	What's planned	What this means
Q3 2026	Enterprise competitor scraping via Bright Data	Turn the manual competitor pricing feature into a fully automatic one
Q4 2026	Enterprise single sign-on (SAML 2.0, OAuth2, Okta)	Let big companies log in through the identity systems they already use, like Okta or Azure AD
Q1 2027	Neural time-series forecasting (DeepAR, Temporal Fusion Transformers)	Add more advanced prediction models on top of Prophet for trickier, more complex demand patterns
Q2 2027	Multi-echelon supply network optimization	Support planning across multiple warehouses at once, instead of one location at a time

How This Was Actually Built

Worth noting as a point of credibility rather than a footnote: this wasn't built as one giant commit. The repository's own history shows nine sequential feature branches, each one following conventional commit style, starting with the basic project scaffolding, then the forecasting engine, then external signals like weather, then the AI reasoning layer, then the EOQ optimizer, then the simulator, then the dashboard itself, then monitoring, and finally production hardening before the v1.0.0 tag. It reads like a real, incrementally shipped product rather than a single dump of code.

Quick Reference

Item	Value
Dashboard	Next.js 14, port 3000
Gateway	Node.js Express, port 4000
Forecasting engine	Python FastAPI, port 8000 (interactive API docs at /docs)

Item	Value
Database	PostgreSQL 15 with TimescaleDB, port 5432
Cache and job queue	Redis 7, port 6379
Monitoring	Prometheus on port 9090, Grafana on port 3001 (admin / admin)
Live demo	fascinating-narwhal-5f1ac9.netlify.app
Start everything locally	cp .env.example .env, then docker compose -f infra/docker-compose.yml up --build -d
Check container health	docker compose -f infra/docker-compose.yml ps